

Face Detection Benchmark: Classical CNN vs Quanvolutional QNN

Stage 1 of a one-shot face recognition system. Faces from LFW, non-faces from CIFAR-10, 32×32 grayscale patches.

0.963 → 0.586

QNN precision, balanced →
1:10 background

0.998 → 0.957

CNN precision, balanced →
1:10 background

404 vs 27

False alarms at 1:10 (QNN vs
CNN)

0.57

Best single-statistic AUC (0.5 =
chance). Benchmark passes the
validity gate

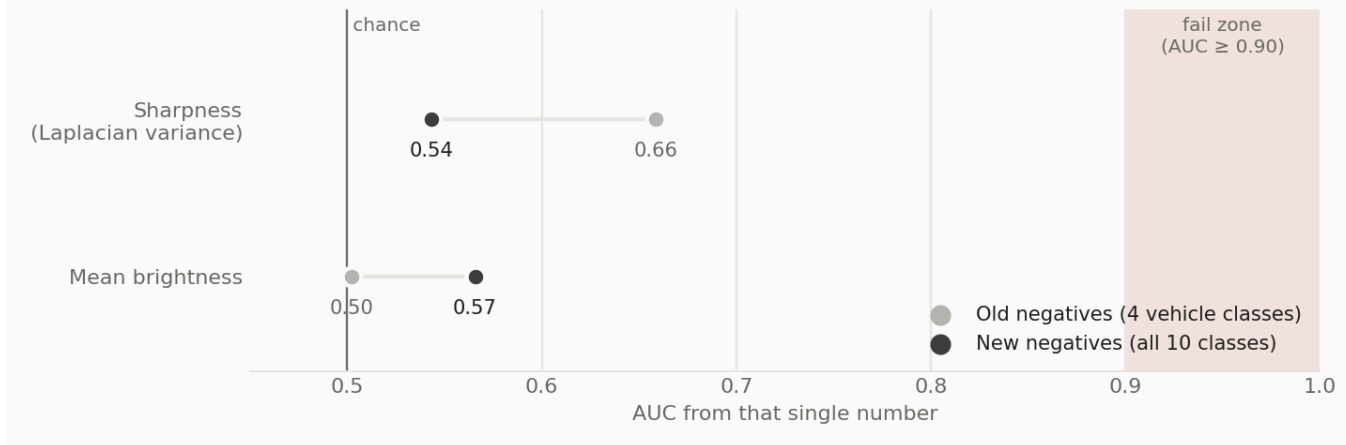
1. Setup

- **Faces:** LFW, every identity with at least 20 photos: 62 people, 3,023 images.
- **Non-faces:** CIFAR-10, all 10 classes, matched 1:1. 80/20 stratified split (4,836 train / 1,210 test), seed 42.
- **CNN:** Conv 3×3 (16 channels) → max-pool → dense 16 → 2 classes. 65,746 trainable parameters.
- **QNN:** 4-qubit quanvolution over 2×2 patches (RY encoding, 2-layer ring of CNOTs, Pauli-Z readout, circuit depth 15) → max-pool → dense 16 → 2 classes. 4,146 trainable parameters plus 24 fixed circuit parameters.

2. Is the benchmark valid? (Gate G0)

Faces and non-faces come from different datasets, so a model could score well by recognising *which dataset* an image came from instead of whether it contains a face. Before any training, three trivial baselines are run. If one number per image (sharpness or brightness) could separate the classes, the accuracy numbers would be meaningless.

Gate G0: no single image statistic separates faces from non-faces

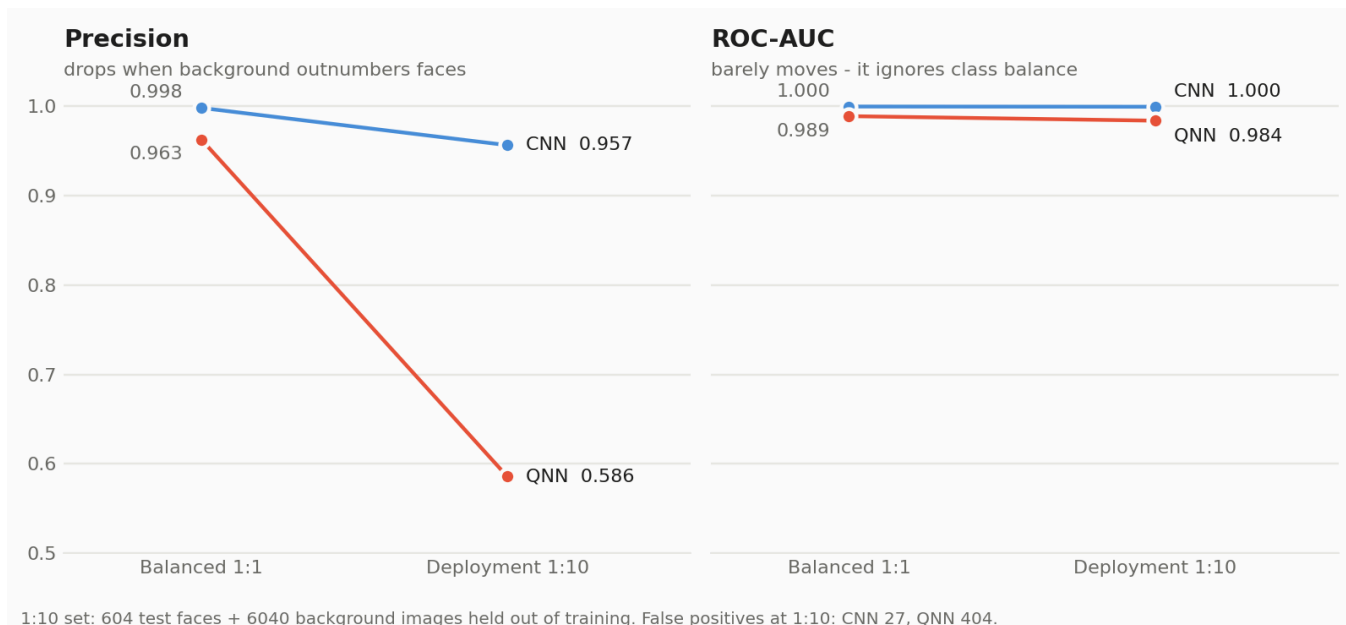


Both statistics stay close to chance. Widening the negatives from 4 vehicle classes to all 10 classes reduced the sharpness signal further (AUC 0.658 → 0.543, Cohen's d 0.63 → 0.29).

A logistic regression on raw pixels does reach AUC 0.992. That is expected for a real task with spatial structure and is not a confound: the gate's self-test (`tests/test_gate_g0.py`) builds two classes with identical sharpness and brightness that differ only in layout, and raw pixels separate those perfectly too.

3. Main result: performance at a realistic class ratio

A sliding-window detector sees far more background than faces. The models were therefore also tested on **604 test faces + 6,040 background images** (1:10). None of the background images were used in training or in the balanced test set.

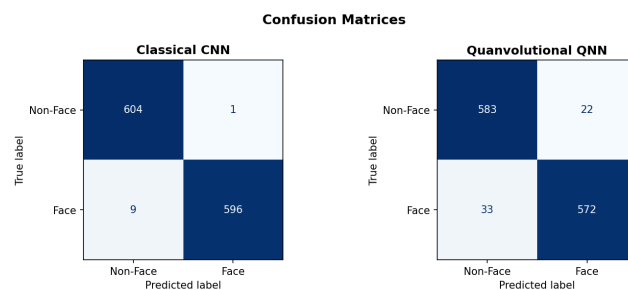
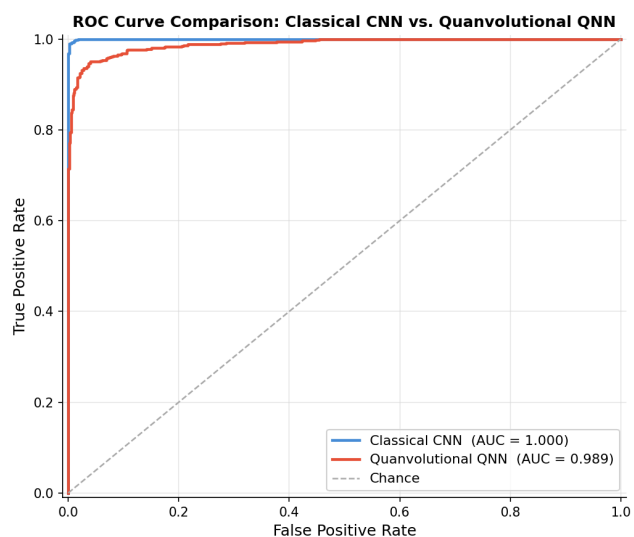


At 1:10	● CNN	● QNN
Precision	0.957	0.586
Recall	0.985	0.945
F1	0.971	0.723
Average precision	0.996	0.920
ROC-AUC	1.000	0.984
False positives (rate)	27 (0.0045)	404 (0.0669)

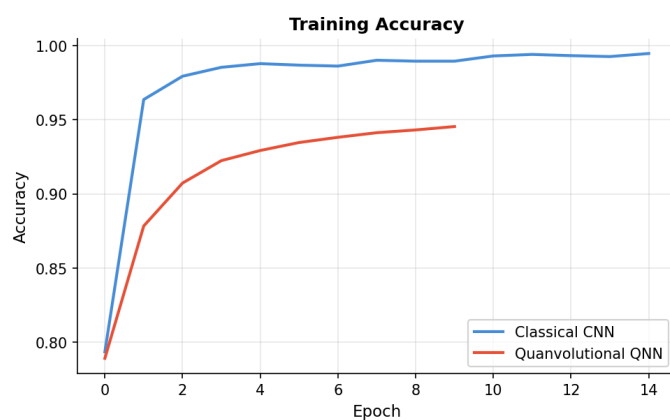
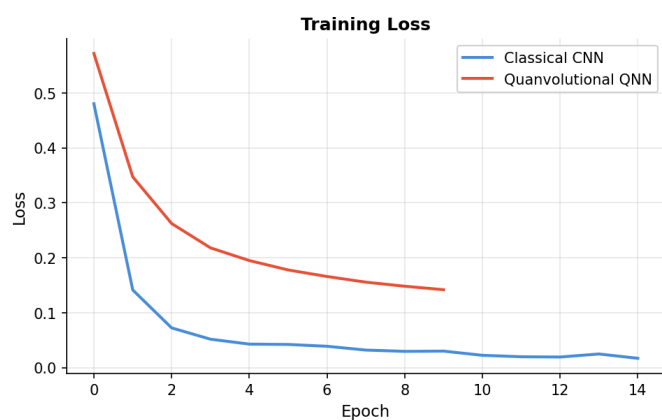
Why ROC-AUC hides this: ROC-AUC depends only on how the model ranks faces against non-faces, not on how many of each there are. Precision does depend on it. The QNN's false-positive rate of 6.7% looks harmless at 1:1 but turns into 404 false alarms when background outnumbered faces ten to one. For a detector, precision and average precision are the numbers to report.

4. Balanced test set (1:1, 1,210 images)

Metric	● CNN	● QNN
Accuracy	0.992	0.955
Precision	0.998	0.963
Recall	0.985	0.945
F1	0.992	0.954
ROC-AUC	1.000	0.989
Trainable parameters	65,746	4,146 (+24 fixed)
Inference per 32×32 image (simulator)	0.76 ms	3,940 ms



Training Progress: CNN vs. QNN



QNN latency is classical simulation time on a CPU, not quantum hardware time.

5. Limitations

- **The quantum circuit is not trained.** Quantum features are computed once and cached to disk, and the classifier trains on the cached tensor, so the loss never reaches the 24 circuit parameters. The QNN is therefore a *fixed random quantum feature* baseline. Its parameters are marked as frozen so the parameter count reflects this.
- **The two models are not capacity-matched.** The CNN's classifier receives 4,096 inputs (16 channels $\times$ 16 $\times$ 16); the QNN's receives 256 (4 channels pooled to 8 $\times$ 8). The CNN also trains for 15 epochs against 10. The accuracy gap cannot yet be attributed to the quantum layer.
- **One seed, one split.** No error bars yet, so small differences should not be read as real.
- **No validation set.** Epoch counts are fixed rather than tuned.

6. Next steps

1. Match capacity: same channel count and pooling into the classifier, same epoch budget for both models.
2. Train the circuit end-to-end (`QuantumNet(freeze_quantum=False)`) and compare against the fixed-circuit result.
3. Repeat over 5 seeds and report means with confidence intervals.
4. Train at the skewed ratio as well; focal loss only becomes relevant then.

7. Reproduce

- `python main.py` : full run (G0 gate, both models, balanced and 1:10 evaluation, all figures). Console output of the reported run is in `outputs/run_log.txt` .
- `python -m src.baseline_g0` : validity gate, old vs new negatives.
- `python tests/test_pipeline.py` and `python tests/test_gate_g0.py` : checks that train, test and background sets never overlap, and that the gate catches known confounds.
- `python make_presentation.py` : rebuilds this page from `outputs/` .